

Sales Data Change Pipeline – Implementation Guide

Overview

This document provides a complete implementation guide for building a Snowflake SCD Type 1 data pipeline using:

- GitHub
- AWS Glue
- Amazon S3
- Amazon SNS
- AWS IAM
- Snowflake Snowpipe
- Snowflake Streams
- Snowflake Stored Procedures
- Snowflake Tasks

The pipeline architecture:

```
GitHub
→ AWS Glue
→ Amazon S3
→ S3 Event Notification
→ Amazon SNS
→ Snowflake Snowpipe
→ Landing Tables
→ Streams
→ Stored Procedure (SCD Type 1 MERGE)
→ Target Table
```

Section 1 – GitHub Repository Setup

Repository Structure

Recommended repository structure:

```
sales_data_change_pipeline/
|
```

```
├── glue-jobs/
│   ├── github-full-data.py
│   └── github-change-data.py
├── snowflake/
│   ├── storage-integration.sql
│   ├── stages.sql
│   ├── tables.sql
│   ├── pipes.sql
│   ├── streams.sql
│   ├── procedures.sql
│   └── tasks.sql
└── sample-data/
    ├── customer_full_data.csv
    └── customer_change_data.csv
```

GitHub Branch Strategy

Recommended branches:

```
main
/dev
feature/full-data-pipeline
feature/change-data-pipeline
fix/snowpipe-errors
```

Section 2 – AWS S3 Bucket Setup

Create S3 Bucket

Bucket name:

```
sales-data-update-with-scdtype1
```

Folder Structure

The Glue jobs automatically create these folders:

```
full-data/  
change-data/
```

Example paths:

```
s3://sales-data-update-with-scdtype1/full-data/  
s3://sales-data-update-with-scdtype1/change-data/
```

S3 Event Notifications

Full Data Notification

Configuration:

```
Name: full-data-event  
Prefix: full-data/  
Suffix: .csv  
Event Type: All object create events  
Destination: SNS Topic  
Topic: github-s3-snowflake-full-data
```

Change Data Notification

Configuration:

```
Name: change-data-event  
Prefix: change-data/  
Suffix: .csv  
Event Type: All object create events  
Destination: SNS Topic  
Topic: github-s3-snowflake-change-data
```

Section 3 – AWS IAM Configuration

Glue IAM Role

Trusted Entity

AWS Service → Glue

Managed Policies

Attach:

AWSGlueServiceRole
AmazonS3FullAccess

Example role name:

glue-github-s3-role

Snowflake IAM Role

Example role name:

SNOWFLAKE-S3-SALES-DATA-ROLE

S3 Permissions

Attach:

AmazonS3FullAccess

Trust Relationship

Updated after Snowflake storage integration creation.

Example trust relationship:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::820663349045:user/rqqi1000-s"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "YOUR_EXTERNAL_ID"
        }
      }
    }
  ]
}
```

Section 4 – AWS SNS Setup

SNS Topics

Create two Standard SNS topics.

Full Data Topic

```
github-s3-snowflake-full-data
```

Change Data Topic

```
github-s3-snowflake-change-data
```

SNS Access Policy – Full Data

```
{
  "Version": "2012-10-17",
  "Id": "SnowflakeSnowpipeSNSPolicy",
  "Statement": [
    {
```

```

    "Sid": "AllowS3BucketToPublish",
    "Effect": "Allow",
    "Principal": {
      "Service": "s3.amazonaws.com"
    },
    "Action": "SNS:Publish",
    "Resource": "arn:aws:sns:us-east-1:655691758300:github-s3-snowflake-full-
data",
    "Condition": {
      "ArnEquals": {
        "aws:SourceArn": "arn:aws:s3:::sales-data-update-with-scdtype1"
      },
      "StringEquals": {
        "aws:SourceAccount": "655691758300"
      }
    }
  },
  {
    "Sid": "AllowSnowflakeToSubscribe",
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::820663349045:user/rqqi1000-s"
    },
    "Action": "SNS:Subscribe",
    "Resource": "arn:aws:sns:us-east-1:655691758300:github-s3-snowflake-full-
data"
  }
]
}

```

Section 5 – AWS Glue Jobs

Glue Job Type

Use:

```

Python Shell
Python Version: 3.9

```

Full Data Glue Job

Purpose:

Download full customer data CSV from GitHub and upload to S3 full-data/

Example Glue Script:

```
import boto3
import requests
from datetime import datetime

S3_BUCKET = "sales-data-update-with-scdtype1"
S3_FOLDER = "full-data/"

GITHUB_URL = "https://raw.githubusercontent.com/manningjtn/
sales_data_change_pipeline/dev/customer_full_data%20%282%29.csv"

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

s3 = boto3.client("s3")

response = requests.get(GITHUB_URL, timeout=60)
response.raise_for_status()

s3_key = f"{S3_FOLDER}customer_full_data_{timestamp}.csv"

s3.put_object(
    Bucket=S3_BUCKET,
    Key=s3_key,
    Body=response.content
)

print(f"Uploaded to s3://{S3_BUCKET}/{s3_key}")
```

Change Data Glue Job

Purpose:

Download incremental customer change CSV from GitHub and upload to S3 change-data/

Example Glue Script:

```
import boto3
import requests
from datetime import datetime

S3_BUCKET = "sales-data-update-with-scdtype1"
S3_FOLDER = "change-data/"

GITHUB_URL = "https://raw.githubusercontent.com/manningjtn/
sales_data_change_pipeline/dev/customer_change_data%20%283%29.csv"

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

s3 = boto3.client("s3")

response = requests.get(GITHUB_URL, timeout=60)
response.raise_for_status()

s3_key = f"{S3_FOLDER}customer_change_data_{timestamp}.csv"

s3.put_object(
    Bucket=S3_BUCKET,
    Key=s3_key,
    Body=response.content
)

print(f"Uploaded to s3://{S3_BUCKET}/{s3_key}")
```

Section 6 – Snowflake Setup

Database and Schema

```
CREATE OR REPLACE DATABASE SALES_DATA_CHANGE;

CREATE OR REPLACE SCHEMA SALES_DATA_CHANGE.CHANGE_SCHEMA;
```

Warehouse

```
CREATE OR REPLACE WAREHOUSE COMPUTE_WH
WITH
WAREHOUSE_SIZE = 'XSMALL'
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE;
```

File Format

```
CREATE OR REPLACE FILE FORMAT CSV_FORMAT
TYPE = CSV
SKIP_HEADER = 1
FIELD_OPTIONALLY_ENCLOSED_BY = ''
EMPTY_FIELD_AS_NULL = TRUE
NULL_IF = ('', 'NULL', 'null');
```

Section 7 – Snowflake Storage Integration and Stage

Storage Integration

```
CREATE OR REPLACE STORAGE INTEGRATION SALES_INT
TYPE = EXTERNAL_STAGE
STORAGE_PROVIDER = 'S3'
ENABLED = TRUE
STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::655691758300:role/SNOWFLAKE-S3-SALES-DATA-
ROLE'
STORAGE_ALLOWED_LOCATIONS = ('s3://sales-data-update-with-scdtype1/');
```

Describe Integration

```
DESC INTEGRATION SALES_INT;
```

Use returned values to update the AWS IAM trust relationship.

Stage

```
CREATE OR REPLACE STAGE SALES_DATA_UPDATE
URL = 's3://sales-data-update-with-scdtype1/'
STORAGE_INTEGRATION = SALES_INT
FILE_FORMAT = CSV_FORMAT;
```

Test Stage Access

```
LIST @SALES_DATA_UPDATE/full-data/;
```

Section 8 – Snowflake Tables and Snowpipes

FULL_DATA Table

```
CREATE OR REPLACE TABLE SALES_DATA_CHANGE.CHANGE_SCHEMA.FULL_DATA (
  CUSTOMERNAME VARCHAR,
  PHONE VARCHAR,
  ADDRESSLINE1 VARCHAR,
  ADDRESSLINE2 VARCHAR,
  CITY VARCHAR,
  STATE VARCHAR,
  POSTALCODE VARCHAR,
  COUNTRY VARCHAR,
  TERRITORY VARCHAR,
  CONTACTFIRSTNAME VARCHAR,
  CONTACTLASTNAME VARCHAR,
  UPDATE_DTS TIMESTAMP_NTZ,
  LOAD_DTS TIMESTAMP_NTZ
);
```

CHANGE_DATA_LANDING Table

```
CREATE OR REPLACE TABLE SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_LANDING (
  CUSTOMERNAME VARCHAR,
```

```
PHONE VARCHAR,  
ADDRESSLINE1 VARCHAR,  
ADDRESSLINE2 VARCHAR,  
CITY VARCHAR,  
STATE VARCHAR,  
POSTALCODE VARCHAR,  
COUNTRY VARCHAR,  
TERRITORY VARCHAR,  
CONTACTFIRSTNAME VARCHAR,  
CONTACTLASTNAME VARCHAR  
);
```

FULL_DATA_PIPE

```
CREATE OR REPLACE PIPE SALES_DATA_CHANGE.CHANGE_SCHEMA.FULL_DATA_PIPE  
AUTO_INGEST = TRUE  
AWS_SNS_TOPIC = 'arn:aws:sns:us-east-1:655691758300:github-s3-snowflake-full-  
data'  
AS  
COPY INTO SALES_DATA_CHANGE.CHANGE_SCHEMA.FULL_DATA  
FROM (  
    SELECT  
        $1 AS CUSTOMERNAME,  
        $2 AS PHONE,  
        $3 AS ADDRESSLINE1,  
        $4 AS ADDRESSLINE2,  
        $5 AS CITY,  
        $6 AS STATE,  
        $7 AS POSTALCODE,  
        $8 AS COUNTRY,  
        $9 AS TERRITORY,  
        $10 AS CONTACTFIRSTNAME,  
        $11 AS CONTACTLASTNAME,  
        CURRENT_TIMESTAMP() AS UPDATE_DTS,  
        CURRENT_TIMESTAMP() AS LOAD_DTS  
    FROM @SALES_DATA_UPDATE/full-data/  
)  
FILE_FORMAT = (FORMAT_NAME = 'CSV_FORMAT')  
ON_ERROR = 'CONTINUE';
```

CHANGE_DATA_PIPE

```
CREATE OR REPLACE PIPE SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_PIPE
AUTO_INGEST = TRUE
AWS_SNS_TOPIC = 'arn:aws:sns:us-east-1:655691758300:github-s3-snowflake-change-
data'
AS
COPY INTO SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_LANDING
FROM (
    SELECT
        $1 AS CUSTOMERNAME,
        $2 AS PHONE,
        $3 AS ADDRESSLINE1,
        $4 AS ADDRESSLINE2,
        $5 AS CITY,
        $6 AS STATE,
        $7 AS POSTALCODE,
        $8 AS COUNTRY,
        $9 AS TERRITORY,
        $10 AS CONTACTFIRSTNAME,
        $11 AS CONTACTLASTNAME
    FROM @SALES_DATA_UPDATE/change-data/
)
FILE_FORMAT = (FORMAT_NAME = 'CSV_FORMAT')
ON_ERROR = 'CONTINUE';
```

Section 9 – Streams, Procedures, and Tasks

Stream

```
CREATE OR REPLACE STREAM SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_STREAM
ON TABLE SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_LANDING
APPEND_ONLY = TRUE;
```

SCD Type 1 Stored Procedure

```
CREATE OR REPLACE PROCEDURE
SALES_DATA_CHANGE.CHANGE_SCHEMA.UPDATE_FULL_DATA_SCD1()
RETURNS VARCHAR
```

```

LANGUAGE JAVASCRIPT
EXECUTE AS OWNER
AS
$$

try {

snowflake.execute({sqlText: `BEGIN TRANSACTION;`});

snowflake.execute({sqlText: `
CREATE OR REPLACE TEMPORARY TABLE
SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_TEMP AS
SELECT
    CUSTOMERNAME,
    PHONE,
    ADDRESSLINE1,
    ADDRESSLINE2,
    CITY,
    STATE,
    POSTALCODE,
    COUNTRY,
    TERRITORY,
    CONTACTFIRSTNAME,
    CONTACTLASTNAME,
    CURRENT_TIMESTAMP(6) AS UPDATE_DTS
FROM SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_STREAM;
`});

snowflake.execute({sqlText: `
MERGE INTO SALES_DATA_CHANGE.CHANGE_SCHEMA.FULL_DATA TGT
USING SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_TEMP TMP
ON TGT.CUSTOMERNAME = TMP.CUSTOMERNAME

WHEN MATCHED THEN UPDATE SET
    TGT.PHONE = TMP.PHONE,
    TGT.ADDRESSLINE1 = TMP.ADDRESSLINE1,
    TGT.ADDRESSLINE2 = TMP.ADDRESSLINE2,
    TGT.CITY = TMP.CITY,
    TGT.STATE = TMP.STATE,
    TGT.POSTALCODE = TMP.POSTALCODE,
    TGT.COUNTRY = TMP.COUNTRY,
    TGT.TERRITORY = TMP.TERRITORY,
    TGT.CONTACTFIRSTNAME = TMP.CONTACTFIRSTNAME,
    TGT.CONTACTLASTNAME = TMP.CONTACTLASTNAME,
    TGT.UPDATE_DTS = TMP.UPDATE_DTS

WHEN NOT MATCHED THEN INSERT
(

```

```

        CUSTOMERNAME,
        PHONE,
        ADDRESSLINE1,
        ADDRESSLINE2,
        CITY,
        STATE,
        POSTALCODE,
        COUNTRY,
        TERRITORY,
        CONTACTFIRSTNAME,
        CONTACTLASTNAME,
        UPDATE_DTS,
        LOAD_DTS
    )
VALUES
(
    TMP.CUSTOMERNAME,
    TMP.PHONE,
    TMP.ADDRESSLINE1,
    TMP.ADDRESSLINE2,
    TMP.CITY,
    TMP.STATE,
    TMP.POSTALCODE,
    TMP.COUNTRY,
    TMP.TERRITORY,
    TMP.CONTACTFIRSTNAME,
    TMP.CONTACTLASTNAME,
    TMP.UPDATE_DTS,
    CURRENT_TIMESTAMP(6)
);
`});

snowflake.execute({sqlText: `COMMIT`;});

return 'SCD Type 1 Procedure Executed Successfully';

}
catch (err)
{
    snowflake.execute({sqlText: `ROLLBACK`;});
    throw 'Error: ' + err;
}

$$;

```

Task

```
CREATE OR REPLACE TASK SALES_DATA_CHANGE.CHANGE_SCHEMA.UPDATE_FULL_DATA_TASK
WAREHOUSE = COMPUTE_WH
SCHEDULE = '1 HOUR'
WHEN
SYSTEM$STREAM_HAS_DATA('SALES_DATA_CHANGE.CHANGE_SCHEMA.CHANGE_DATA_STREAM')
AS
CALL SALES_DATA_CHANGE.CHANGE_SCHEMA.UPDATE_FULL_DATA_SCD1();
```

Enable Task

```
ALTER TASK SALES_DATA_CHANGE.CHANGE_SCHEMA.UPDATE_FULL_DATA_TASK RESUME;
```

Monitor Task History

```
SELECT *
FROM TABLE(
    INFORMATION_SCHEMA.TASK_HISTORY(
        TASK_NAME => 'UPDATE_FULL_DATA_TASK',
        RESULT_LIMIT => 10
    )
);
```

Final Architecture

```
GitHub
→ AWS Glue
→ Amazon S3
→ S3 Event Notification
→ Amazon SNS
→ Snowflake Snowpipe
→ Landing Table
→ Stream
→ SCD Type 1 Stored Procedure
```

- FULL_DATA Target Table
- Snowflake Task

This implementation provides a complete cloud-native SCD Type 1 ingestion and update pipeline using AWS and Snowflake.